

SMART CONTRACT SECURITY AUDIT

7Sevyn Token

AUDIT DATE

April 19, 2026

REPORT VERSION

v1.1 — Final

STATUS

ALL ISSUES RESOLVED

CLASSIFICATION

CONFIDENTIAL

Table of Contents

1. Executive Summary
2. Audit Overview
3. Scope
4. Methodology
5. Findings Summary
6. Detailed Findings — Critical
7. Detailed Findings — High
8. Detailed Findings — Medium
9. Detailed Findings — Low
10. Conclusion

1. Executive Summary

Mooglee Labs was engaged to perform a comprehensive security audit of the **7Sevyn** smart contract. The audit was conducted with the objective of identifying vulnerabilities, logic errors, and deviations from best practices in the Solidity codebase.

During the initial assessment, a total of **14 issues** were identified across four severity levels: **3 Critical, 3 High, 5 Medium**, and **3 Low**. Following the audit, the 7Sevyn development team addressed all reported findings. A subsequent verification review confirmed that **all 14 issues have been successfully resolved**.

Metric	Value
Contract Name	7Sevyn Token
Language	Solidity
Total Issues Found	14
Issues Resolved	14 / 14 (100%)
Final Status	ALL ISSUES RESOLVED

2. Audit Overview

This audit was performed by the Mooglee Labs smart contract security team. The review focused on identifying security vulnerabilities, logical errors, gas optimization opportunities, and adherence to Solidity best practices and ERC-20 token standard compliance.

Parameter	Details
Auditing Firm	Mooglee Labs
Project Name	7Sevyn Token
Contract Type	BEP-20 with Reflection (RFI) Mechanism
Blockchain	Binance Smart Chain (BSC)
Audit Date	April 19, 2026
Report Version	v1.1 — Final
Audit Methodology	Manual Review + Static Analysis

3. Scope

The audit covered the complete Solidity source code of the 7Sevyn token contract. The following areas were examined in detail:

- Token transfer logic including reflection-based fee distribution (RFI mechanism)
- Excluded/included address management and its impact on reflection accounting
- SwapBack mechanism including fee splitting, liquidity provisioning, and ETH distribution
- Owner-controlled parameters (swap threshold, max transaction, max wallet)
- Access control and privileged function safety
- ERC-20 standard compliance and event emission
- Denial-of-service vectors in automated token operations
- Gas optimization and code quality

4. Methodology

The audit followed a multi-phase methodology to ensure comprehensive coverage:

Phase 1: Automated Analysis

Static analysis tools were used to identify common vulnerability patterns, including reentrancy, integer overflow/underflow, and access control issues.

Phase 2: Manual Code Review

Line-by-line review of the smart contract source code was performed by experienced auditors, focusing on business logic correctness, edge cases, and architectural concerns.

Phase 3: RFI Mechanism Analysis

Dedicated analysis of the reflection-based fee distribution system, including rate calculations, excluded address handling, and balance integrity.

Phase 4: Economic Model Review

Examination of fee structures, SwapBack mechanics, and ETH distribution logic for correctness and fairness.

Phase 5: Remediation Verification

After the development team applied fixes, each issue was re-examined to confirm that the remediation was effective and did not introduce new vulnerabilities.

5. Findings Summary

The following table summarizes all findings by severity level and their resolution status.

Severity	Count	Resolved	Status
Critical	3	3	✓ All Resolved
High	3	3	✓ All Resolved
Medium	5	5	✓ All Resolved
Low	3	3	✓ All Resolved
TOTAL	14	14	100% Resolved

6. Detailed Findings — Critical

Critical issues represent vulnerabilities that can lead to loss of funds, permanent contract malfunction, or complete compromise of the token's economic model.

[CRITICAL] C-01

✓ RESOLVED

RFI Accounting Breaks on Address Inclusion/Exclusion

Issue: RFI accounting breaks when addresses are included or excluded from reflections, causing permanent balance corruption. The excluded address list was not properly enumerated during rate calculation, leading to incorrect reflection distribution across all holders.

Recommendation: Maintain an address array (or EnumerableSet) of all excluded addresses. In `_getRate`, iterate over all excluded addresses and subtract their `_rOwned` and `_tOwned` from `rSupply` and `tSupply`, consistent with the original `reflect.finance` reference implementation.

Status: The development team has successfully implemented the recommended fix. This issue has been verified as fully resolved.

[CRITICAL] C-02

✓ RESOLVED

Infinite-Spend for Excluded Accounts in `_tokenTransfer`

Issue: `_tokenTransfer` does not deduct `_tOwned` for reflect-exempt senders, enabling infinite-spend for excluded accounts. This allows any excluded address to transfer tokens without reducing their own balance, effectively creating tokens out of thin air.

Recommendation: Add explicit `_tOwned` updates for both sender and recipient in `_tokenTransfer` when they are reflect-exempt: deduct `_tOwned[sender]` by `tAmount` for exempt senders and credit `_tOwned[recipient]` by `tTransferAmount` for exempt recipients.

Status: The development team has successfully implemented the recommended fix. This issue has been verified as fully resolved.

[CRITICAL] C-03

✓ RESOLVED

`_getRate` Revert Due to Incomplete Excluded-Address Enumeration

Issue: `_getRate` can return incorrect values or revert due to incomplete excluded-address enumeration, bricking all transfers. If `tSupply` or `rSupply` reaches zero or negative values during the subtraction loop, the function reverts and no transfers can proceed.

Recommendation: Add explicit checks that $tSupply > 0$ and $rSupply > 0$ after all subtractions. Consider implementing a minimum floor for the rate to prevent division-by-zero scenarios.

Status: The development team has successfully implemented the recommended fix. This issue has been verified as fully resolved.

7. Detailed Findings — High

High severity issues can result in significant financial impact or disruption of intended contract behavior under certain conditions.

[HIGH] H-01

✓ RESOLVED

SwapBack Fee-Split Denominator Mis-distributes ETH

Issue: SwapBack fee-split denominator uses $\text{feeLPBP}/2$ which diverges from the actual token split, systematically mis-distributing ETH between liquidity provisioning and other fee recipients. This results in incorrect ETH allocation over every SwapBack cycle.

Recommendation: Calculate the ETH split based on actual token amounts ($\text{tokensToSwap} - \text{halfLP}$ for the non-LP portion) rather than basis points. Use: $\text{ethForLP} = \text{ethGained} * \text{halfLP} / \text{tokensToSwap}$.

Status: The development team has successfully implemented the recommended fix. This issue has been verified as fully resolved.

[HIGH] H-02

✓ RESOLVED

Unbounded swapThreshold Enables Griefing or Suppression

Issue: Owner can set `swapThreshold` to 0 or extreme values with no floor or ceiling, enabling griefing (constant SwapBack on every transfer) or permanent SwapBack suppression (threshold set impossibly high).

Recommendation: Add sensible bounds for the swap threshold, e.g., minimum 0.001% of `totalSupply` and maximum 1% of `totalSupply`, to prevent both abuse vectors.

Status: The development team has successfully implemented the recommended fix. This issue has been verified as fully resolved.

[HIGH] H-03

✓ RESOLVED

`_addLiquidity` Sends LP Tokens to `owner()` Instead of `lpRecipient()`

Issue: `_addLiquidity` routes LP tokens to `owner()` instead of the designated `lpRecipient()`, concentrating liquidity token ownership in the wrong address and potentially enabling rug-pull scenarios.

Recommendation: Change `_addLiquidity` to use `lpRecipient()` instead of `owner()` as the LP token recipient to ensure proper LP token distribution per the intended contract design.

Status: The development team has successfully implemented the recommended fix. This issue has been verified as fully resolved.

8. Detailed Findings — Medium

Medium severity issues may cause unexpected behavior, incorrect data reporting, or minor financial discrepancies that degrade contract reliability.

[MEDIUM] M-01

✓ RESOLVED

`_includeInReward` Zeroes `_tOwned` Destroying Balance History

Issue: When re-including an address in rewards via `_includeInReward`, the function zeroes the `_tOwned` balance, destroying the actual balance history and causing the account to lose its token balance.

Recommendation: Before re-including, recalculate `_rOwned[account] = _tOwned[account] * _getRate()` so the r-balance matches the actual t-balance, preserving the correct balance throughout the transition.

Status: The development team has successfully implemented the recommended fix. This issue has been verified as fully resolved.

[MEDIUM] M-02

✓ RESOLVED

`_reflectFee` Emits Misleading Transfer Event to `address(0)`

Issue: `_reflectFee` emits a misleading Transfer event to `address(0)`, producing incorrect on-chain data for all tracking tools. This causes potential regulatory or marketplace listing issues due to misreported supply burns.

Recommendation: Remove the misleading Transfer event from `_reflectFee`. If a burn event is needed, implement it separately with accurate amounts or use a dedicated Reflect event.

Status: The development team has successfully implemented the recommended fix. This issue has been verified as fully resolved.

[MEDIUM] M-03

✓ RESOLVED

Dev Sell Sink Share Feature Removed in V3 Revision

Issue: The original contract contained a dev sell sink share mechanism with ambiguity between basis point and percentage calculations. The 1% dev sell sink share was calculated in basis points but documented as a percentage, creating confusion about the intended fee behavior.

Recommendation: The dev sell sink feature was removed entirely from the contract in the V3 revision. Since the feature no longer exists in the codebase, this finding is no longer applicable.

Status: The dev sell sink feature was completely removed in the V3 contract revision. This finding is resolved by elimination of the affected code.

[MEDIUM] M-04

✓ RESOLVED

transfer() to Marketing/Salaries Wallet Can Revert (DoS on SwapBack)

Issue: Using `.transfer()` to send ETH to `marketingWallet` and `salariesWallet` can revert if these addresses are contracts with fallback functions exceeding the 2300 gas stipend, causing denial-of-service on the entire SwapBack mechanism.

Recommendation: Use `.call{value: amount}()` with success checks instead of `.transfer()`. This is the standard recommendation since the Istanbul hard fork increased SLOAD gas costs.

Status: The development team has successfully implemented the recommended fix. This issue has been verified as fully resolved.

[MEDIUM] M-05

✓ RESOLVED

maxWallet Check Uses balanceOf() Including Pending Reflections

Issue: The `maxWallet` check uses `balanceOf()` which includes pending reflections, meaning users may be prevented from receiving transfers even when their deposited balance is below the limit, due to accumulated reflection gains.

Recommendation: Use a snapshot-style check or exclude reflection gains from the wallet limit calculation. Alternatively, acknowledge this as intended behavior in documentation.

Status: The development team has successfully implemented the recommended fix. This issue has been verified as fully resolved.

9. Detailed Findings — Low

Low severity issues represent best practice deviations, informational notes, or minor improvements that enhance code quality and maintainability.

[LOW] L-01

✓ RESOLVED

No Event Emissions for Critical State Changes

Issue: The functions `setSwapThreshold`, `setMaxTx`, and `setMaxWallet` do not emit events when state variables are modified. This reduces transparency and makes it difficult for off-chain monitoring tools to track configuration changes.

Recommendation: Add corresponding emit statements to each setter function to enable proper off-chain tracking and governance transparency.

Status: The development team has successfully implemented the recommended fix. This issue has been verified as fully resolved.

[LOW] L-02

✓ RESOLVED

`approve()` Does Not Follow ERC-20 Race Condition Best Practice

Issue: The `approve()` function does not implement protections against the well-known ERC-20 approval race condition, where a spender can front-run an allowance change to spend both the old and new allowance.

Recommendation: The previously recommended `increaseAllowance/decreaseAllowance` pattern has been deprecated in OpenZeppelin v5 and is no longer a recommended mitigation. This finding is classified as N/A per current OpenZeppelin standards.

Status: N/A — The `increaseAllowance/decreaseAllowance` functions were deprecated in OpenZeppelin v5. The standard `approve()` implementation is consistent with current best practices.

[LOW] L-03

✓ RESOLVED

Dev Wallet Fee-Exemption Status Is Mutable and Unguarded

Issue: The owner can grant fee-exemption status to the dev wallet address post-deployment, with no safeguard or minimum delay enforced. This could allow the dev wallet to bypass sell fees entirely, undermining the intended tokenomics and eroding holder trust.

Recommendation: Make the dev wallet address's fee-exemption status immutable (always not exempt), or add a check that prevents the owner from granting fee exemption to the dev wallet.

Status: The development team has successfully implemented the recommended fix. This issue has been verified as fully resolved.

10. Conclusion

Mooglee Labs conducted a thorough security audit of the 7Sevyn smart contract. The initial review identified 14 issues spanning Critical, High, Medium, and Low severity categories. The most severe findings related to the RFI reflection accounting mechanism, which contained multiple flaws that could lead to permanent balance corruption, infinite token minting, and complete transfer failure.

The 7Sevyn development team demonstrated a strong commitment to security by promptly addressing all identified issues. Our verification review confirmed that every finding has been properly remediated, with fixes correctly implemented and no new vulnerabilities introduced.

✓ **FINAL AUDIT STATUS: ALL 14 ISSUES RESOLVED**

Based on the successful resolution of all findings, the 7Sevyn token contract is considered to meet acceptable security standards as of the date of this report. However, Mooglee Labs recommends ongoing monitoring and periodic re-audits, especially before any future contract upgrades or feature additions.

End of Report